

SKY BET PRO · MONETISATION ASSESSMENT

Scan economics and how to charge for them

The scanner already meters itself against The Odds API. This turns that meter into a billed product — what a scan really costs, how many \$5 users each API tier carries, and the exact changes to make in this codebase.

3 September 2026 · TanStack Start · Supabase · Cloudflare · The Odds API v4

THE DIRECT ANSWER

15M credits, 3 scans a day, \$5 a head

On the **15M plan at \$249/month**, with every user capped at 3 scans a day and a scan costing what your code costs it today — one league across the `eu,uk` regions, so 2 credits — the plan carries **83,333 paying users**. That is the floor, not the ceiling: the shared server cache means real capacity is several times higher.

USERS ON THE 15M PLAN

83,333

3 scans/day × 30 days × 2 credits = 180 credits per user per month. Cache ignored.

API COST OF ONE SCAN

\$0.000033

\$249 ÷ 15,000,000 credits, ×2. A \$5 user would need 150,000 scans to cost you \$5.

USERS TO COVER THE \$249

50

At \$5/month. Past 50 subscribers the API

line stops mattering.

THE CONCLUSION THAT MATTERS

Data is not your cost problem. At scale the API is under **0.1% of revenue**, while payment processing is **9–15%**. So the scan meter should not exist to recover API spend. It exists to stop one account burning the shared quota, and to give people a reason to move up a tier. Price it for behaviour, not for cost.

ASSESSMENT

A “scan” is not one credit

The Odds API bills `markets × regions` per odds request, and every 10 named bookmakers counts as one region. Your scanner already computes this at `src/routes/scanner.tsx:679` — and the default of `eu,uk` means the cheapest scan you currently ship is two credits, not one. A date scan multiplies that by the number of leagues selected.

SCAN SHAPE	CREDITS	WHERE IT COMES FROM
League scan, <code>eu,uk</code> Current default	2	1 league call × 2 regions
League scan, one region Cheapest useful setting	1	1 league call × 1 region
League scan, ≤10 named books	1	<code>ceil(books / 10)</code> replaces regions
Date scan, 5 leagues, <code>eu,uk</code>	10	5 parallel calls × 2 regions
Date scan, 12 leagues, <code>eu,uk</code>	24	12 parallel calls × 2 regions
League list <code>/sports</code>	0	Free endpoint, and cached 6h anyway
Any scan served from cache	0	3-minute shared memo in <code>odds-server.ts</code>

THE CACHE IS YOUR REAL MARGIN

The memo in `src/lib/odds-server.ts` is keyed on the request URL, not the user, and it coalesces concurrent calls. Two hundred people scanning the Premier League in the same three minutes cost you **one** upstream call. Every capacity number here assumes zero cache benefit; a realistic 70% hit rate multiplies them by 3.3×.

FORCE-REFRESH IS THE LEAK

`refresh: true` deliberately bypasses that cache, and it is a button any user can hold down. Nothing rate-limits it today. Whatever billing you build, force-refresh must cost more than a normal scan and be capped per hour.

MODEL IT

Capacity calculator

Move the inputs to see how many paying users each Odds API tier carries, and what the data costs you as a share of revenue.

ODDS API PLAN

100K — 100,000 — \$59/mo



SCANS PER USER PER DAY



LEAGUES PER SCAN



1 = the League tab. Higher = the Date tab, which fires one call per selected league.

REGIONS PER CALL



Your default is `eu,uk` — 2.

CACHE HIT RATE



Share of scans served from the shared memo, costing nothing upstream.

PRICE PER USER / MONTH



\$5

PAYING USERS THIS PLAN CARRIES

555

100,000 credits ÷ 180 credits per user per month

CREDITS PER SCAN	CREDITS / USER / MO	REVENUE AT CAPACITY	API AS % OF REVENUE
2	180	\$2,778	2.12%
BREAK-EVEN USERS	API COST PER SCAN		
12	\$0.00118		

Every tier at these settings — paying users carried, log scale

Starter	2 users
20K	111 users
100K	555 users
5M	27,777 users
15M	83,333 users
100M	555,555 users
	1 10 100 1K 10K 100K 1M 10M

THE PROBLEM WITH THE PLAN AS STATED

You cannot sell one scan at a time

Two things break “\$5/month buys one scan, extra scans cost a fee”.

One scan a month is not a product. A user who scans once and finds nothing has bought nothing, and churns in week one after you paid to

acquire them. One scan is a demo — give it away free, and make \$5 buy a month of daily use.

Per-scan charges lose money on fees. Every card charge carries a fixed component of roughly \$0.30. Selling a scan for \$0.20 costs \$0.30 to collect. Anything charged per scan has to be batched — into a prepaid pack, or into one monthly invoice.

CHARGE	FEE	YOU KEEP	TAKE RATE	VERDICT
\$0.20 single scan	\$0.31	−\$0.11	155%	LOSES MONEY
\$5.00 monthly	\$0.45	\$4.55	8.9%	FEE-HEAVY
\$9.00 scan pack	\$0.56	\$8.44	6.2%	WORKABLE
\$50.00 annual	\$1.75	\$48.25	3.5%	BEST

Modelled at 2.9% + \$0.30 (Stripe standard). A merchant of record such as Paddle or Lemon Squeezy runs 5% + \$0.50 — on a \$5 charge that is a 15% take rate, which is why annual billing matters more the smaller your monthly price is.

RECOMMENDATION

Sell scan credits, keep \$5 as the floor

Define one unit — a **scan credit is one league call** — and price everything in it. A League-tab scan spends 1. A Date-tab scan over 5 leagues spends 5. That is honest, it maps exactly to what the upstream API charges you, and it makes heavy multi-league users self-select into a higher tier with no extra logic.

Trial

Free · once

- 10 scan credits, no card
- Calculator and saved

Starter

\$5 /mo

- 40 credits — about 1 a day

Pro

\$12 /mo

- 150 credits — about 5 a day

bets

- Cross-book odds locked

- League scans, one region
- Up to 3 leagues per date scan
- \$50/year — two months free

- Cross-book mode, all regions
- Up to 10 leagues per date scan
- 20 force-refreshes a month

Sharp

\$29 /mo

- 600 credits
- Unlimited leagues per date scan
- Unmetered force-refresh
- PDF export, priority support

TOP-UPS, NOT PER-SCAN BILLING

When credits run out, sell a pack rather than a scan: **50 for \$4, 150 for \$9, 500 for \$25**. One charge, one fee, one decision. Subscription credits expire at period end; purchased packs never expire — that asymmetry is what makes packs feel fair and subscriptions feel urgent.

IF YOU WANT LITERAL PAY-PER-SCAN

There is one way to charge per scan without the fee problem: Stripe metered billing. A \$5/month licensed price includes 40 credits, then overage accrues at \$0.10 a credit and settles in the *same monthly invoice*. One charge a month, unlimited granularity, and it honours your original model exactly. It is more moving parts — meter events, proration, mid-cycle upgrades — so ship packs first and switch if users ask.

CHOOSING A PROCESSOR

--- - - - -

What can actually collect \$5 from your users

The right rail depends on two things only you can decide: where your business entity is, and where your users are. If they are in Sub-Saharan Africa, card penetration is low and **mobile money is the difference between a 5% and a 40% conversion rate on a \$5 subscription.**

RAIL	COLLECTS FROM	FEE ON \$5	RECURRING	USE IT WHEN
Stripe Billing + metered usage	Global cards	\$0.45	Native, excellent	You have a US/UK/EU entity, or Stripe Atlas
Paddle / Lemon Squeezy Merchant of record	Global cards, PayPal	\$0.75	Native	You want them to own tax and pay out to your country
Flutterwave Cards + MTN / Airtel money	Most of Africa, global cards	~\$0.15 local	Tokenised, your logic	Your users are in Zambia, Uganda, Ghana, Kenya, Nigeria
Paystack Cards + bank transfer	NG, GH, KE, ZA, CI	~\$0.10 local	Native plans	Your entity is in one of those five countries
Manual + proof What you have today	Anyone, any method	\$0	Admin does it	Keep as a fallback — never as the only path

GET THE MERCHANT CATEGORY RIGHT BEFORE YOU BUILD

Paddle and Lemon Squeezy prohibit gambling outright, and Stripe restricts it. Sky Bet Pro is an *analytics tool* — it prices cover bets, it does not take stakes or pay out — which normally passes review. But say exactly that in your application, up front, in your own words. An account frozen after launch costs far more than a careful week on the application.

KEEP THE PROOF-OF-PAYMENT PATH ALIVE

Your `/pending` upload and the admin queue in `src/routes/admin.tsx` already work. Do not delete them — they become the manual rail for users who pay by mobile money or bank transfer outside the processor. An admin then grants credits with one ledger insert instead of flipping `status`.

IMPLEMENTATION · IN THIS ORDER

Seven changes to this codebase

The order matters. The ledger and the server-side choke point must exist and be correct before a single payment is taken, or you will be reconciling by hand later.

01 Put the balance in an append-only ledger

supabase/2026-09-03-billing.sql

Never a mutable `scans_left` column. A ledger makes every grant, spend and refund auditable, makes webhook replays harmless, and lets expiry be a property of the grant row rather than a nightly job.

```
-- one row per credit movement; the balance is their sum
create table public.scan_ledger (
  id          bigserial primary key,
  user_id     uuid not null references auth.users on delete cascade,
  delta       int  not null,          -- +40 grant, -1 spend
  reason      text not null,          -- subscription|topup|scan|refund|manual
  expires_at  timestamptz,           -- null = never (packs)
  source_id   text,                  -- provider event id
  meta        jsonb,
  created_at  timestamptz default now()
);
create unique index on public.scan_ledger (source_id) where source_id is not null;
create index on public.scan_ledger (user_id, created_at desc);

create view public.scan_balance as
select user_id, sum(delta)::int as balance
from public.scan_ledger
where expires_at is null or expires_at > now();
```

```

group by user_id;

create table public.subscriptions (
  user_id          uuid primary key references auth.users on delete cascade
  provider         text not null,
  customer_id      text,
  subscription_id  text unique,
  plan            text not null default 'starter',
  status           text not null,    -- active|past_due|canceled|trialing
  current_period_end timestamptz,
  updated_at       timestamptz default now()
);

```

RLS: a user may `select` their own ledger rows and subscription, and nothing else. No client may `insert` or `update` either table. Writes happen only through the function below and the webhook's service role.

02 Make spending atomic and unfakeable

supabase/2026-09-03-billing.sql

A `security definer` function that checks and debits in one transaction. It reads `auth.uid()`, so it works when called with the user's own token — exactly the pattern `assertApprovedUser` already uses.

```

create or replace function public.consume_scan(p_cost int, p_meta jsonb default null)
returns int language plpgsql security definer as $$
declare v_bal int;
begin
  select coalesce(sum(delta),0) into v_bal
  from public.scan_ledger
  where user_id = auth.uid()
  and (expires_at is null or expires_at > now());

  if v_bal < p_cost then
    raise exception 'INSUFFICIENT_SCANS';
  end if;

  insert into public.scan_ledger(user_id, delta, reason, meta)
  values (auth.uid(), -p_cost, 'scan', p_meta);

  return v_bal - p_cost;
end $$;

```

03 Debit at the only choke point that exists

src/lib/odds-server.ts

This is the security-critical step. `oddsFn` is an RPC endpoint — any signed-in user can call it directly with their own payload, bypassing every control in `scanner.tsx`. The debit must happen server-side, and the cost must be recomputed from the request rather than trusted from the client.

```
// replaces assertApprovedUser at its call sites in odds-api.ts
export async function assertCanScan(accessToken: string, cost: number) {
  const client = clientFor(accessToken); // the existing pattern
  const { data } = await client.auth.getUser(accessToken);
  if (!data.user) throw new Error("Your session has expired – sign in again.");

  const { data: profile } = await client
    .from("profiles").select("role").eq("id", data.user.id).single();
  if (profile?.role === "admin") return { balance: Infinity }; // never bill

  const { data: bal, error } = await client.rpc("consume_scan", {
    p_cost: cost,
    p_meta: { at: new Date().toISOString() },
  });
  if (error?.message.includes("INSUFFICIENT_SCANS"))
    throw new Error("You're out of scans. Top up or upgrade to keep scanning.");
  if (error) throw new Error(error.message);
  return { balance: bal as number };
}
```

Charge **one credit per league call**, and skip the debit when the same user gets the same URL back from cache inside the TTL — a small `Map<userId + url, timestamp>` beside the existing `memo`. Double-clicks and page reloads then cost nobody anything while a genuine second look still pays.